

AI BlogNest API

AI-Powered Blog Management and Content Generation REST API Platform

PHASE 3: PROJECT DESIGN

Role	Team Member
Team Leader	Praveen Kumar M
Team Member 1	Prabu R
Team Member 2	Muralidharan S
Team Member 3	Yadhava Moorthy Y

Submitted as a phase-wise project submission.

1. System Architecture

The AI BlogNest API employs a layered Model-View-Controller (MVC) system design. Decoupling the client interface from database logic and third-party generative services ensures high system modularity and ease of maintenance.



2. Main Components

- **Express Gateway (`app.js`):** Handles HTTP request routing, CORS headers, JSON body parsing, and request logging middleware.
- **Middleware Layer:** Executes stateless token verification via JWT, enforces role-based access limits, and handles input validation.
- **Controller Layer:** Contains primary domain logic for authentication, blog management, comment workflows, and prompt processing.
- **Gemini AI Service (`geminiService.js`):** Handles API integration with Google's `@google/generative-ai` SDK for automated text generation and article summarization.
- **Mongoose Data Models:** Defines schema blueprints and data integrity constraints for database persistence.

3. Project Structure Evidence

```
AI-BLOG-NEST
├── node_modules
├── src
│   ├── config
│   └── db.js
```

```

├── controllers
│   ├──
│   ├── aiController.js
│   ├──
│   ├── authController.js
│   ├── errorMiddleware.js
│   ├── blogController.js
│   └── models
│       ├── Blog.js
│       ├── routes
│       ├── User.js
│       ├── aiRoutes.js
│       ├──
│       ├── authRoutes.js
│       ├──
│       ├── geminiService.js
│       └── blogRoutes.js
├── .env
├──
├── .gitignore
├── app.js
├──
├── package.json
├──

```

Figure 3.1: Backend Project Directory Structure (VS Code)

4. User Flow

1. **Authentication:** Client submits credentials to `/api/auth/login` . Upon verification, server returns a signed JWT.
2. **Authorized Request:** Client sends requests with `Authorization:Bearer <token>` attached to request headers.
3. **Middleware Verification:** Express middleware validates token signatures and checks user roles against endpoint requirements.
4. **Execution & Response:** Controller executes requested CRUD or Gemini AI task, interacting with MongoDB or AI SDKs, and returns formatted JSON.

5. Data Entitles

UserSchema

- `_id` : ObjectId (Primary Key, Auto-generated)
- `name` : String (Required, Trimmed)
- `email` : String (Required, Unique, Lowercase)
- `password` : String (Required, Hashed with bcrypt)
- `role` : String (Enum: ['admin', 'editor', 'author', 'reader'], Default: 'reader')

Blog Post Schema

- `_id` : ObjectId (Primary Key, Auto-generated)
- `title` : String (Required, Trimmed)
- `content` : String (Required)
- `category` : String (Required, Indexed)

- author :ObjectId(Ref:User Schema, Required)
- authorName:String (Required)
- createdAt updatedAt :Date (Timestamps)